



**POLISH-JAPANESE ACADEMY
OF INFORMATION TECHNOLOGY**

Wydział Informatyki

Katedra Systemów Inteligentnych i Data Science

Inteligentne systemy przetwarzania danych

Mateusz Jastrzębski

Nr albumu s27397

**Zastosowanie metod uczenia maszynowego do
prognozowania danych rynkowych na danych BTC**

Praca inżynierska

Dr. inż. Adam Szmigielski

Warszawa, czerwiec, 2026

Warszawa, 30.06.2026 r.

(data, miejscowość)

OŚWIADCZENIE STUDENTA DOTYCZĄCE WYKORZYSTANIA SZTUCZNEJ INTELIGENCJI (AI)

Ja, niżej podpisany/a: Mateusz Jastrzębski

(imię i nazwisko)

student/ka kierunku: Informatyka (Inteligentne systemy przetwarzania danych)

(nazwa kierunku)

numer albumu studenta: s27397

Oświadczam, że zapoznałem/am się z treścią Zarządzenia Rektora Polsko-Japońskiej Akademii Technik Komputerowych z dnia 1 kwietnia 2026 r. oraz z „Wytycznymi dotyczącymi wykorzystywania sztucznej inteligencji w procesie dydaktycznym PJATK”, stanowiącymi załącznik do ww. zarządzenia.

Oświadczam, że ewentualne wykorzystanie narzędzi sztucznej inteligencji (AI) w niniejszej pracy miało charakter wyłącznie pomocniczy i nie stanowiło zastępstwa dla mojej samodzielnej pracy.

Oświadczam, że treść pracy stanowi moje własne, samodzielne opracowanie oraz ponoszę pełną odpowiedzialność za jej zawartość, niezależnie od wykorzystania narzędzi AI.

Oświadczam, że nie wykorzystywałem/am narzędzi AI do generowania treści stanowiących zasadniczą część pracy, w szczególności analiz, wniosków, interpretacji ani części podlegających ocenie jako przejaw twórczości własnej.

W przypadku korzystania z narzędzi AI sporządziłem/am Raport użycia AI zgodnie z obowiązującymi Wytycznymi i dołączyłem/am go do pracy.

.....

(czytelny podpis studenta/studentki)

Contents

Streszczenie	5
Abstract	5
Wstęp.....	7
1 Podstawy teoretyczne	9
1.1 Charakter problemu: predykcja na danych czasowych	9
1.2 Dane wejściowe i ich format	9
1.3 Definicja targetu i horyzontu predykcji	11
1.4 Walidacja czasowa i unikanie data leakage	11
1.5 Metryki i sposób raportowania wyników	12
1.6 Zakres i ograniczenia pracy	13
2 Technologie i pipeline	14
2.1 Wybór źródła danych	15
2.2 Stos technologiczny i środowisko	15
2.3 Struktura projektu i organizacja kodu	16
2.4 Projekt pipeline'u ML wg Jansena	17
2.5 Narzędzia do kontroli eksperymentów	19
3 Implementacja i wyniki	20
3.1 Założenia eksperymentu i wybrany wariant danych	20
3.2 Implementacja pozyskania i przygotowania danych	20
3.3 Implementacja budowy cech i targetu	21
3.4 Schemat walidacji czasowej w kodzie	21
3.5 Modele bazowe i porównywane modele	22
3.6 Strojenie hiperparametrów	23
3.7 Porównanie modeli	24
3.8 Kalibracja prawdopodobieństw i strojenie progu decyzyjnego	26

3.8.1 Kalibracja izotoniczna	26
3.8.2 Strojenie progu decyzyjnego	27
3.9 Analiza stabilności w czasie i typowe błędy	29
3.9.1 Uproszczona symulacja strategii inwestycyjnej.....	29
3.9.2 Stabilność modeli per reżim rynkowy	30
3.10 Podsumowanie części praktycznej, ograniczenia i dalsze kroki	31
Podsumowanie	32
Bibliografia.....	33
Spis rysunków	34
Spis tabel	34

Streszczenie

Praca analizuje praktyczne możliwości zastosowania klasycznych modeli uczenia maszynowego w zadaniu klasyfikacji binarnej kierunku zwrotu z Bitcoina w horyzoncie pięciu dni. Zaimplementowano kompletny potok przetwarzania danych: pobieranie notowań z giełdy Binance, generowanie dziesięciu cech inżynierskich, walidację walk-forward z pięcioma foldami oraz porównanie pięciu modeli (dwa modele typu baseline'y persistence, regresja logistyczna, las losowy, XGBoost). Standardową ewaluację uzupełniono o kalibrację izotoniczną prawdopodobieństw, strojenie progu decyzyjnego, symulację strategii inwestycyjnej z kosztami transakcyjnymi oraz analizę stabilności w pięciu reżimach rynkowych. Modele osiągają istotną przewagę nad baseline'ami w metryce ROC-AUC, jednak ich przewaga ekonomiczna nad strategią kup-i-trzymaj pozostaje, w badanym okresie silnego trendu wzrostowego, ograniczona.

Słowa kluczowe: uczenie maszynowe, klasyfikacja binarna, Bitcoin, walk-forward, kalibracja izotoniczna

Abstract

The contemporary cryptocurrency market, with particular focus on Bitcoin, attracts growing interest from both retail investors and the research community, which attempts to apply machine learning methods to predict the direction of price changes. This thesis analyses the practical possibilities of applying classical supervised learning models to the binary classification task of predicting the direction of the BTC return over a five-day horizon. The thesis implements a complete data processing pipeline covering the acquisition of historical quotations from the Binance exchange, the generation of ten engineered features, walk-forward validation with five chronological folds, and the training and comparison of five models: two persistence baselines and three classification models - logistic regression, random forest and XGBoost. Standard evaluation is complemented by four extensions: isotonic calibration of probabilities, tuning of the decision threshold, a simplified simulation of an investment strategy with transaction costs, and analysis of result stability across five manually defined market regimes. The results confirm that machine learning models achieve a statistically meaningful advantage over the baselines in the ROC-AUC metric, however their economic advantage over a simple buy-and-hold strategy is, in the studied period of strong upward trend, limited.

Keywords: machine learning, binary classification, Bitcoin, walk-forward, isotonic calibration

Wstęp

Uczenie maszynowe (ang. machine learning, ML) jest jedną z najszybciej rozwijających się dziedzin współczesnej informatyki, znajdującą zastosowanie między innymi w wizji komputerowej, rozpoznawaniu mowy, medycynie i finansach. W odróżnieniu od klasycznego programowania, w którym człowiek definiuje precyzyjne reguły działania programu, modele uczenia maszynowego wnioskują zależności wprost z danych, samodzielnie odkrywając wzorce, które mogłyby być trudne do sformalizowania w postaci jawnych reguł. Ta cecha sprawia, że uczenie maszynowe jest szczególnie atrakcyjne w problemach o dużej złożoności i zaszumieniu, do których należy między innymi prognozowanie szeregów czasowych na rynkach finansowych.

Rynki finansowe stanowią od wielu lat naturalne pole eksperymentalne dla metod uczenia maszynowego. Dane rynkowe są publicznie dostępne, ich pozyskiwanie nie wymaga zaawansowanej infrastruktury, a powtarzalne metryki ekonomiczne, takie jak stopa zwrotu lub współczynnik Sharpe'a, pozwalają jednoznacznie porównywać jakość różnych podejść. Jednocześnie szeregi czasowe stóp zwrotu są wyjątkowo trudnym przypadkiem aplikacyjnym ze względu na bardzo niski stosunek sygnału do szumu, niestacjonarność procesu generującego dane oraz silną autokorelację kolejnych obserwacji [1].

W ostatniej dekadzie szczególnie dużo uwagi poświęca się rynkowi kryptowalut, w którym największym kapitalizacyjnie aktywem pozostaje Bitcoin (BTC). Bitcoin charakteryzuje się wysoką zmiennością cenową, całodobową płynnością oraz brakiem powiązania z klasycznymi fundamentami makroekonomicznymi, co sprawia, że prognozowanie jego stóp zwrotu na podstawie czystych danych technicznych jest zarówno interesujące badawczo, jak i wymagające metodologicznie. Niniejsza praca podejmuje próbę odpowiedzi na pytanie, jakie wyniki da się realnie uzyskać przy uczciwie przeprowadzonej walidacji modelu uczenia maszynowego prognozującego kierunek zwrotu z Bitcoina.

Cel pracy. Głównym celem niniejszej pracy jest zaprojektowanie, implementacja i empiryczna ewaluacja kompletnego potoku przetwarzania danych do prognozowania kierunku zmian ceny Bitcoina w horyzoncie pięciu dni, opartego na klasycznych metodach uczenia maszynowego nadzorowanego. Praca koncentruje się na rozwiązaniu praktycznych problemów metodycznych związanych z modelowaniem szeregów czasowych, ze

szczególnym uwzględnieniem zagadnień związanych z wyciekami danych z przyszłości i niestacjonarnością procesów rynkowych.

Zakres pracy. Zakres pracy został świadomie ograniczony do dziennych danych OHLCV pary handlowej BTC/USDT z giełdy Binance, w okresie od stycznia 2019 roku do kwietnia 2026 roku. Praca nie obejmuje danych intraday, danych alternatywnych typu sentyment z mediów społecznościowych ani modeli z rodziny deep learning, które wymagają znacznie większej liczby obserwacji. Symulacja strategii inwestycyjnej przedstawiona w rozdziale trzecim jest świadomie uproszczona i nie zastępuje pełnego silnika backtestowego, czyli kompletnego systemu symulacji strategii na danych historycznych, modelującego m.in. mikrostrukturę rynku i poślizg cenowy (ang. slippage).

Struktura pracy. Praca jest podzielona na trzy zasadnicze rozdziały. Rozdział pierwszy przedstawia podstawy teoretyczne pracy: charakterystykę problemu, strukturę danych wejściowych, definicję zmiennej objaśnianej, zagadnienia walidacji czasowej, metryki ewaluacji oraz zakres i ograniczenia. Rozdział drugi opisuje stos technologiczny zastosowany w części praktycznej: wybór źródła danych, biblioteki Pythona, organizację kodu w repozytorium, projekt pipeline'u inspirowany pracą Jansena oraz narzędzia do kontroli eksperymentów. Rozdział trzeci zawiera prezentację rezultatów: założenia eksperymentu, implementację każdej z warstw potoku, porównanie pięciu modeli, strojenie hiperparametrów, kalibrację i strojenie progów, analizę stabilności w czasie oraz podsumowanie z dyskusją dalszych kierunków.

1 Podstawy teoretyczne

W tym rozdziale przedstawiono fundamenty teoretyczne niezbędne do zrozumienia projektowych i implementacyjnych decyzji podjętych w dalszych rozdziałach pracy. Omówione zostały: charakterystyka problemu prognozowania kierunku zmian cen jako zadania klasyfikacji binarnej, struktura i interpretacja danych OHLCV, konstrukcja zmiennej objaśnianej, koncepcja walidacji walk-forward i zagadnienie wycieku danych z przyszłości, miary jakości predykcji wykorzystane do porównania modeli oraz świadome ograniczenia przyjętego zakresu.

1.1 Charakter problemu: predykcja na danych czasowych

Szereg czasowy to ciąg obserwacji uporządkowanych chronologicznie. W odróżnieniu od klasycznych zbiorów danych w uczeniu maszynowym, kolejne obserwacje szeregu są silnie skorelowane i nie spełniają założenia niezależności i identycznych rozkładów (i.i.d.), co wymaga specjalnego podejścia do procedur uczenia i walidacji [2]. Sygnały finansowe charakteryzują się dodatkowo niestacjonarnością - rozkład zwrotów zmienia się w czasie, a wzorce skuteczne w jednym reżimie rynkowym mogą przestać działać w kolejnym.

Prognozowanie szeregu czasowego można formalizować na dwa równoważne sposoby: jako zadanie regresyjne, w którym celem jest przewidywanie wartości ciągłej (np. stopy zwrotu lub ceny zamknięcia), albo jako zadanie klasyfikacyjne, w którym celem jest przewidywanie kategorii, takiej jak kierunek zmian ceny. W niniejszej pracy zdecydowano się na drugi sposób, z trzech istotnych powodów. Po pierwsze, sygnał w danych dziennych jest na tyle słaby, że model regresyjny ma tendencję do uczenia się predykcji bliskiej średniej, co prowadzi do trywialnych, nieużytecznych prognoz. Po drugie, decyzje inwestycyjne są z natury dyskretne - inwestor podejmuje binarną decyzję o zajęciu lub niezajęciu pozycji, a nie inwestuje część kapitału proporcjonalną do przewidywanej stopy zwrotu. Po trzecie, metryki klasyfikacyjne, takie jak pole pod krzywą ROC i log-loss, lepiej różnicują modele o słabym sygnale niż błędy regresyjne w postaci średniego błędu kwadratowego.

1.2 Dane wejściowe i ich format

Standardową formą reprezentacji danych rynkowych w analizie technicznej jest świeczka OHLCV, zawierająca pięć wartości dla każdego interwału czasowego: cenę otwarcia (Open),

maksymalną cenę w interwale (High), minimalną cenę w interwale (Low), cenę zamknięcia (Close) oraz wolumen obrotu (Volume). Dane OHLCV stanowią podstawowy nośnik informacji dostępnej publicznie dla niemal każdego aktywa finansowego i pozwalają budować szerokiej rodziny cech pochodnych [1].

W przypadku rynku kryptowalut Bitcoin handlowany jest w trybie ciągłym, 24 godziny na dobę, 7 dni w tygodniu, co eliminuje zagadnienie luki cenowej między sesjami obecnej na rynkach klasycznych. Wartości OHLCV mogą być pobierane w dowolnym interwale czasowym - od jednej minuty po jeden tydzień. W niniejszej pracy wybrano interwał dzienny, jako kompromis pomiędzy rozdzielczością informacji a liczbą dostępnych obserwacji.

Surowe ceny OHLCV nie są, w większości przypadków, bezpośrednio przydatne jako wejście do modelu uczenia maszynowego. Są one niestacjonarne, a ich skala zmienia się o rzędy wielkości w czasie. Model uczony bezpośrednio na surowych cenach testowany byłby na wartościach spoza treningu (np. ceny powyżej 100 tysięcy dolarów, których nie było w danych uczących), co czyni je bezużytecznymi jako wejście. Z tego powodu w analizie technicznej i finansowym uczeniu maszynowym stosuje się inżynierię cech (ang. feature engineering), której celem jest transformacja danych OHLCV w zestaw stacjonarnych, znormalizowanych zmiennych objaśniających.

W niniejszej pracy zaimplementowano dziesięć cech, podzielonych na pięć rodzin:

- Zwroty logarytmiczne - logarytm naturalny ze stosunku ceny zamknięcia kolejnych obserwacji, $\log(\text{close}(t)/\text{close}(t-1))$. Zwroty logarytmiczne są preferowane wobec zwrotów procentowych ze względu na ich addytywność w czasie i bardziej symetryczny rozkład [1].
- Momentum - kumulatywne zwroty w oknach 5, 10 i 20 dni, definiowane jako $\text{close}(t)/\text{close}(t-n) - 1$. Cechy te oddają kierunek i siłę średnio- i krótkoterminowego trendu.
- Zmienność - odchylenie standardowe zwrotów logarytmicznych w oknach 10 i 20 dni. Wysokie wartości tej cechy odzwierciedlają okresy wysokiej niepewności rynkowej.
- RSI 14 - wskaźnik siły względnej (Relative Strength Index) w 14-dniowym oknie, klasyczny oscylator analizy technicznej. RSI przyjmuje wartości w zakresie $[0, 100]$ i wskazuje na warunki wykupienia lub wyprzedania rynku.

- Średnie kroczące i ich stosunek - proste średnie kroczące ceny zamknięcia w oknach 20 i 50 dni (SMA 20, SMA 50) oraz ich stosunek SMA20/SMA50, który pełni rolę uproszczonego sygnału trendu.

1.3 Definicja celu i horyzontu predykcji

Dla danego horyzontu prognozy h zmienna objaśniana zdefiniowana jest jako:

$$y(t) = 1 \text{ jeżeli } \text{close}(t + h) > \text{close}(t), \text{ w przeciwnym razie } y(t) = 0$$

W niniejszej pracy przyjęto horyzont $h = 5$ dni, co odpowiada jednemu tygodniowi handlowemu i stanowi naturalną jednostkę czasu w analizie rynkowej. Krótsze horyzonty są silnie zaszumione, dłuższe zaś tracą warunek przybliżonej niezależności kolejnych obserwacji.

Zmienna objaśniana w przyjętym sformułowaniu zależy wyłącznie od relacji pomiędzy cenami zamknięcia $\text{close}(t)$ oraz $\text{close}(t+5)$. Pięciodniowy horyzont oznacza, że ostatnich pięciu obserwacji w zbiorze nie można oznaczyć etykietą - są one usuwane przed fazą modelowania. Jest to jeden z dwóch krytycznych punktów, w których mogłoby dojść do wycieku danych z przyszłości, gdyby procedura nie została starannie zaprojektowana (drugi punkt omówiono w sekcji 1.4).

1.4 Walidacja czasowa i unikanie wycieku danych

Centralnym zagadnieniem metodologicznym pracy jest poprawna walidacja modeli na danych czasowych. Standardowa procedura walidacji krzyżowej (k-fold cross-validation), powszechnie stosowana w klasycznych problemach uczenia maszynowego, opiera się na założeniu niezależności obserwacji. W przypadku szeregów czasowych założenie to nie jest spełnione, a zastosowanie losowego podziału prowadzi do sytuacji, w której zbiór treningowy może zawierać obserwacje czasowo późniejsze niż zbiór testowy. Jest to klasyczny przypadek tak zwanego wycieku danych z przyszłości (ang. lookahead bias, data leakage) [2].

Właściwą procedurą walidacji szeregów czasowych jest walidacja walk-forward (lub forward chaining) i polega ona na podziale danych na nakładające się chronologiczne foldy, w których zbiór treningowy zawsze poprzedza zbiór testowy [1]. W praktyce stosowane są dwa warianty

tej procedury: walidacja z oknem rozszerzającym się (ang. expanding window), w której zbiór treningowy w każdym kolejnym foldzie jest powiększany o nowe obserwacje, oraz walidacja z oknem przesuwającym się (ang. sliding window), w której oba zbiory mają stałą długość. W niniejszej pracy zastosowano wariant z oknem rozszerzającym się, dostarczany przez klasę `TimeSeriesSplit` z biblioteki `scikit-learn`, z parametrem `n_splits = 5`.

Niezależnie od wybranej procedury walidacji, krytycznym aspektem implementacji jest zagwarantowanie, że żaden krok wstępnego przetwarzania danych nie wykorzystuje informacji z przyszłości. W szczególności, standaryzacja cech (skalowanie do średniej zero i odchylenia jednostkowego) musi być dopasowana wyłącznie na podstawie zbioru treningowego bieżącego foldu, a następnie zastosowana do zbioru testowego. Naruszenie tej zasady, na przykład poprzez dopasowanie skamera na całym zbiorze przed podziałem na foldy, prowadzi do nieoczywistego, ale istotnego wycieku informacji statystycznej z przyszłości. W zaimplementowanym potoku zapobiega temu mechanizm `Pipeline` biblioteki `scikit-learn`, omówiony w rozdziale trzecim.

1.5 Metryki i sposób raportowania wyników

Do oceny jakości modeli zastosowano cztery komplementarne miary, każda akcentująca inny aspekt jakości predykcji.

Dokładność (accuracy) - udział obserwacji prawidłowo sklasyfikowanych w całkowitej liczbie obserwacji. Najprostsza z miar, jednak wrażliwa na niezbalansowanie klas i nieuwzględniająca pewności predykcji modelu.

Miara F1 - średnia harmoniczna precyzji i czułości, lepiej oddająca jakość predykcji w obecności niezbalansowanych klas. F1 osiąga maksymalną wartość 1 dla doskonałego klasyfikatora i wartość 0 w przypadku, gdy model nie wykrył ani jednej obserwacji klasy pozytywnej.

Pole pod krzywą ROC (ROC-AUC) - jedna z najczęściej stosowanych miar w klasyfikacji binarnej, niezależna od wyboru progu decyzyjnego. ROC-AUC interpretowana jest jako prawdopodobieństwo, że losowo wybrana obserwacja klasy pozytywnej otrzyma wyższe

przewidywanie prawdopodobieństwa niż losowo wybrana obserwacja klasy negatywnej. Wartość 0,5 odpowiada losowemu zgadywaniu, 1,0 - klasyfikatorowi doskonałemu.

Log-loss (logarytmiczna funkcja straty) - jedyna z czterech miar, która bezpośrednio penalizuje nieprawidłowo skalibrowane prawdopodobieństwa. Log-loss osiąga wartość 0 dla doskonałego klasyfikatora wystawiającego prawdopodobieństwa równe 0 lub 1 w odpowiednich kierunkach; wartość około 0,69 odpowiada klasyfikatorowi zawsze zwracającemu prawdopodobieństwo 0,5.

Łącznie te cztery miary dają pełniejszy obraz jakości modelu niż jakakolwiek z nich brana z osobna. Dokładność i F1 zależą od wyboru progu decyzyjnego (domyślnie 0,5); ROC-AUC i log-loss są od niego niezależne i mierzą jakość samych rankingów prawdopodobieństw.

Każdy model jest ewaluowany w sposób zagregowany - wartości metryk z pięciu foldów walidacji walk-forward są uśredniane i raportowane wraz z odchyleniem standardowym, co pozwala ocenić zarówno przeciętną jakość modelu, jak i stabilność jego wyników między różnymi okresami testowymi.

1.6 Zakres i ograniczenia pracy

Niniejsza praca koncentruje się na opracowaniu i empirycznej ocenie kompletnego potoku uczenia maszynowego dla problemu klasyfikacji binarnej kierunku zwrotu z Bitcoina. Zakres pracy obejmuje:

- Pobieranie i wstępne przetwarzanie dziennych danych OHLCV BTC/USDT z giełdy Binance za okres od stycznia 2019 do kwietnia 2026.
- Konstrukcję dziesięciu cech inżynierskich z surowych danych OHLCV.
- Walidację walk-forward z pięcioma foldami chronologicznymi, opakowaną w mechanizm Pipeline biblioteki scikit-learn dla zagwarantowania braku wycieku danych.
- Trening i porównanie pięciu modeli klasyfikacyjnych: dwóch baseline'ów persistence oraz regresji logistycznej, lasu losowego i XGBoost.
- Cztery rozszerzenia metodyczne: kalibracja izotoniczna prawdopodobieństw, strojenie progu decyzyjnego, uproszczona symulacja strategii inwestycyjnej z kosztami

transakcyjnymi oraz analiza stabilności w pięciu ręcznie zdefiniowanych reżimach rynkowych.

- Pełne śledzenie eksperymentów w narzędziu MLflow w hierarchicznej strukturze parent-child runs.

Praca świadomie nie obejmuje następujących elementów:

- **Danych intraday i modeli sekwencyjnych.** Praca opiera się wyłącznie na dziennych danych OHLCV. Modele z rodziny deep learning, takie jak LSTM lub Transformer, wymagają znacznie większej liczby obserwacji i nie zostały włączone do badań.
- **Wielu aktywów i konstrukcji portfela.** Analiza jest ograniczona do pary BTC/USDT - najpopularniejszego instrumentu kryptowalutowego.
- **Pełnego silnika backtestowego** – kompletnego systemu symulacji strategii na danych historycznych. Symulacja strategii inwestycyjnej w sekcji 3.9 jest świadomie uproszczona i nie modeluje mikrostruktury rynku, poślizgu cenowego (ang. slippage) ani zarządzania portfelem.
- **Danych alternatywnych.** Praca nie wykorzystuje danych sentymentowych z mediów społecznościowych, danych on-chain ani danych makroekonomicznych.

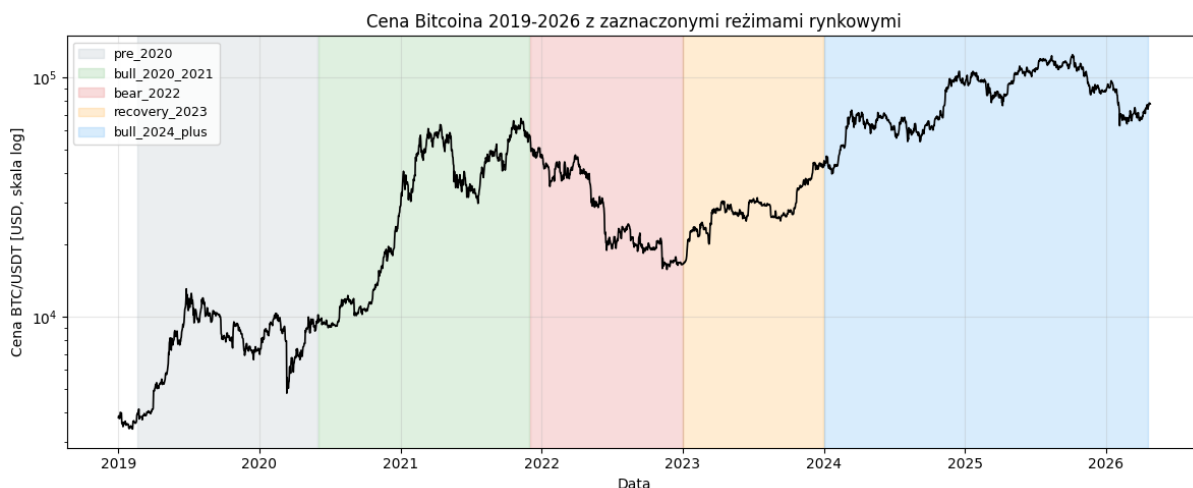
2 Technologie i pipeline

Rozdział drugi przedstawia stos technologiczny zastosowany w części praktycznej pracy. Omówiono wybór źródła danych, użyte biblioteki Pythona, organizację kodu w repozytorium, projekt pipeline'u uczenia maszynowego inspirowany pracą Jansena [1] oraz narzędzia do kontroli eksperymentów. Celem prezentacji jest udokumentowanie decyzji technicznych, które stanowią podstawę implementacji opisanej w rozdziale trzecim.

2.1 Wybór źródła danych

Jako źródło historycznych danych rynkowych wybrano publiczne API giełdy Binance - największej kapitalizacyjnie giełdy kryptowalut na świecie. Wybór ten jest podyktowany trzema czynnikami praktycznymi. Po pierwsze, API Binance jest dostępne bez konieczności uwierzytelnienia w trybie tylko-do-odczytu danych historycznych. Po drugie, giełda oferuje dane od początku roku 2017 dla pary BTC/USDT, co zapewnia wystarczająco długi okres do analizy obejmujący kilka pełnych cykli rynkowych. Po trzecie, dane Binance są powszechnie używane w literaturze i badaniach branżowych, co ułatwia porównywalność wyników z innymi pracami.

Para handlowa BTC/USDT została wybrana jako podstawowy instrument analizy ze względu na najwyższą płynność wśród par bitcoinowych, co minimalizuje wpływ zaszumionych transakcji oraz luk w danych. W okresie objętym analizą (od stycznia 2019 do kwietnia 2026) dane nie zawierają luk czasowych, co jest charakterystyczne dla rynku kryptowalut funkcjonującego w trybie ciągłym.



Rysunek 1. Cena Bitcoina w analizowanym okresie z zaznaczonymi reżimami rynkowymi

2.2 Stos technologiczny i środowisko

Cała implementacja została wykonana w języku Python w wersji 3.11. Wybór tego języka był podyktowany bogactwem ekosystemu bibliotek do uczenia maszynowego oraz analizy

danych finansowych, a także dobrą dokumentacją i wsparciem społeczności. W tabeli 1 zestawiono najważniejsze biblioteki wykorzystane w pracy wraz z ich rolami w potoku.

Tabela 1. Główne biblioteki wykorzystane w pracy

Biblioteka	Wersja	Rola w pracy
pandas	2.3	Manipulacja danych, operacje na DataFrame'ach
numpy	2.4	Obliczenia numeryczne, operacje na wektorach
scikit-learn	1.8	TimeSeriesSplit, Pipeline, regresja logistyczna, las losowy, kalibracja
xgboost	3.2	Implementacja gradient boosting drzew decyzyjnych
ta	0.11	Wskaźniki techniczne (RSI 14 z konwencją TA-Lib)
python-binance	1.0	Klient API giełdy Binance, pobieranie danych historycznych
mlflow	3.11	Śledzenie eksperymentów, hierarchia parent-child runs
matplotlib	3.10	Generowanie wykresów i wizualizacji
pyarrow	23.0	Obsługa formatu Apache Parquet

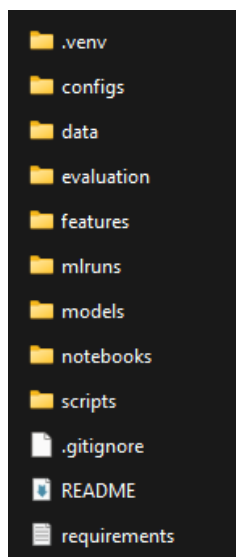
Środowisko zostało skonfigurowane w postaci wirtualnego środowiska Pythona (venv), co zapewnia izolację zależności od systemowej instalacji Pythona. Cała procedura instalacji i uruchomienia jest udokumentowana w pliku README.md repozytorium kodu.

2.3 Struktura projektu i organizacja kodu

Kod projektu został zorganizowany w sposób modułowy, w którym każdy etap przetwarzania jest realizowany przez osobny moduł programistyczny o jasno zdefiniowanym interfejsie.

Modułowy charakter potoku pozwala uruchamiać poszczególnych etapów niezależnie oraz

ułatwia wprowadzanie zmian, na przykład podmianę modelu klasyfikacyjnego, bez konieczności modyfikacji pozostałych komponentów.



Rysunek 2. Struktura repozytorium btc-ml-pipeline

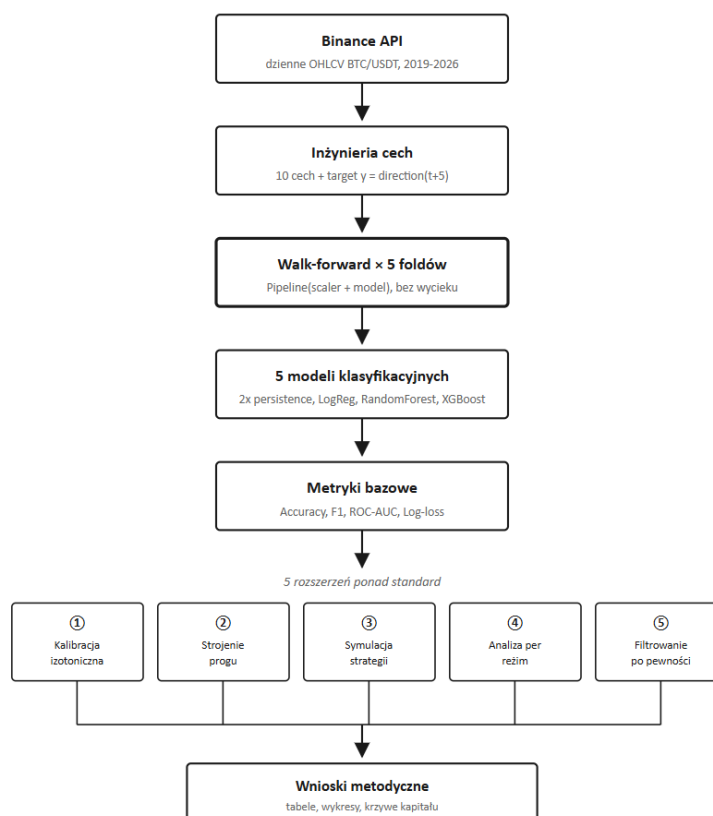
Repozytorium podzielone jest na sześć głównych pakietów. Pakiet data zawiera moduły odpowiedzialne za akwizycję surowych danych oraz katalogi z plikami parquet (raw i processed). Pakiet features zawiera moduły inżynierii cech, definiujące transformacje surowych danych OHLCV na zestaw cech wejściowych modelu. Pakiet models zawiera implementacje pięciu modeli klasyfikacyjnych w postaci zunifikowanego interfejsu zwracającego parę (nazwa modelu, estymator). Pakiet evaluation zawiera moduły walidacji walk-forward, kalibracji, symulacji strategii inwestycyjnej oraz analizy reżimów. Pakiet scripts zawiera skrypty orkiestrujące cały potok od pobrania danych po wygenerowanie wykresów. Pakiet configs zawiera pliki konfiguracyjne specyfikujące hiperparametry i parametry uruchomienia.

Wszystkie zależności zewnętrzne są zadeklarowane w pliku requirements.txt, a środowisko jest reprodukowalne na innym komputerze w czasie poniżej trzech minut, co zostało zweryfikowane w testach końcowych.

2.4 Projekt pipeline'u ML wg Jansena

Architektura potoku uczenia maszynowego została zaprojektowana zgodnie z siedmiokrokowym schematem zaproponowanym przez Stefana Jansena w pracy Machine Learning for Algorithmic Trading [1]. Schemat ten obejmuje kolejno: definicję problemu,

akwizycję danych, inżynierię cech, podział danych, wybór modelu, walidację i ocenę. W pracy schemat ten został rozszerzony o cztery dodatkowe elementy: kalibrację prawdopodobieństw, strojenie progu decyzyjnego, symulację strategii inwestycyjnej oraz analizę stabilności w reżimach rynkowych.



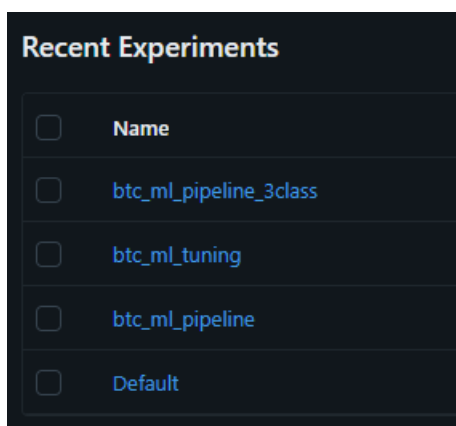
Rysunek 3. Diagram pipeline'u uczenia maszynowego

System składa się z pięciu głównych warstw funkcjonalnych. Warstwa akwizycji danych odpowiada za pobieranie historycznych notowań z giełdy Binance i ich zapis w lokalnym formacie kolumnowym Parquet. Warstwa inżynierii cech realizuje transformację surowych danych OHLCV w zestaw dziesięciu cech inżynierskich oraz konstruuje zmienną objaśnianą zgodnie ze wzorem z sekcji 1.3. Warstwa walidacji implementuje procedurę walk-forward z pięcioma foldami chronologicznymi, opakowując każdy krok preprocessingu i modelowania w obiekt klasy Pipeline biblioteki scikit-learn. Warstwa modelowania zawiera implementację pięciu modeli klasyfikacyjnych. Wreszcie warstwa ewaluacji rozszerzonej realizuje cztery rozszerzenia metodyczne opisane w rozdziale trzecim.

2.5 Narzędzia do kontroli eksperymentów

Do śledzenia eksperymentów wybrano narzędzie MLflow [6] - branżowy standard w obszarze MLOps, używany między innymi w platformie Databricks. MLflow umożliwia automatyczne wersjonowanie hiperparametrów, metryk i artefaktów, a także porównywanie wyników różnych runów w interfejsie graficznym.

W pracy zastosowano hierarchiczną strukturę śledzenia: dla każdego modelu tworzony jest jeden parent run zawierający agregowane metryki (średnia i odchylenie z pięciu foldów), oraz pięć child runów, każdy odpowiadający jednemu foldowi walidacji walk-forward. Taka struktura pozwala w interfejsie MLflow zarówno porównanie wyników na poziomie modeli, jak i analizę stabilności pojedynczego modelu między foldami.



Rysunek 4. Struktura śledzenia eksperymentów w MLflow

Łącznie w trzech eksperymentach MLflow (btc_ml_pipeline, btc_ml_tuning, btc_ml_pipeline_3class) zarejestrowano około siedemdziesięciu runów zawierających pełne metryki, hiperparametry oraz artefakty w postaci wykresów PNG i plików Parquet z surowymi predykcjami.

3 Implementacja i wyniki

Rozdział trzeci stanowi zasadniczą część empiryczną pracy. Opisano w nim założenia eksperymentu, implementację każdej z warstw potoku oraz wyniki pięciu modeli klasyfikacyjnych. Standardowa ewaluacja została uzupełniona o cztery rozszerzenia: kalibrację izotoniczną prawdopodobieństw, strojenie progów decyzyjnych, uproszczoną symulację strategii inwestycyjnej z kosztami transakcyjnymi oraz analizę stabilności wyników w pięciu ręcznie zdefiniowanych reżimach rynkowych. Rozdział kończy podsumowanie z dyskusją ograniczeń i dalszych kierunków rozwoju.

3.1 Założenia eksperymentu i wybrany wariant danych

Eksperyment opiera się na dziennych danych OHLCV pary BTC/USDT z giełdy Binance, obejmujących okres od 1 stycznia 2019 do 25 kwietnia 2026. Po przetworzeniu surowych danych przez moduł inżynierii cech zbiór zawiera 2618 użytecznych obserwacji (z 2672 pobranych), po usunięciu pierwszych wierszy bez pełnych wartości okien przesuwających oraz ostatnich pięciu wierszy bez zdefiniowanej etykiety.

Rozkład klasy pozytywnej (kierunek zwrotu 5-dniowego w górę) wynosi 54,1%, co oznacza, że klasa jest jedynie nieznacznie niebalansowana - typowa proporcja dla rynków rozwijających się w trendzie wzrostowym. Ręcznie zdefiniowano pięć reżimów rynkowych: okres przedpandemiczny (n=468), hossa 2020-2021 (n=548), bessy 2022 (n=396), regeneracja 2023 (n=365) oraz hossa 2024+ (n=841). Reżimy te wykorzystane są wyłącznie w analizie stabilności (sekcja 3.9) i nie wchodziły w skład cech wejściowych modelu.

3.2 Implementacja pozyskania i przygotowania danych

Moduł akwizycji danych odpowiada za pobranie historycznych dziennych notowań pary handlowej BTC/USDT z publicznego API giełdy Binance. Wykorzystano bibliotekę `python-binance`, która nie wymaga uwierzytelnienia w trybie tylko-do-odczytu danych historycznych. Dane są pobierane partiami po 500 świeczek, scalane do pojedynczego obiektu `DataFrame` biblioteki `pandas` i zapisywane do pliku w formacie kolumnowym `Apache Parquet`, zapewniającym szybki odczyt w kolejnych etapach potoku.

W rezultacie modułu otrzymuje się tabelę o 2672 wierszach (jeden wiersz na dzień handlu) i sześciu kolumnach: timestamp, open, high, low, close i volume. Surowe dane nie zawierają luk czasowych dzięki ciągłości handlu na rynku kryptowalut.

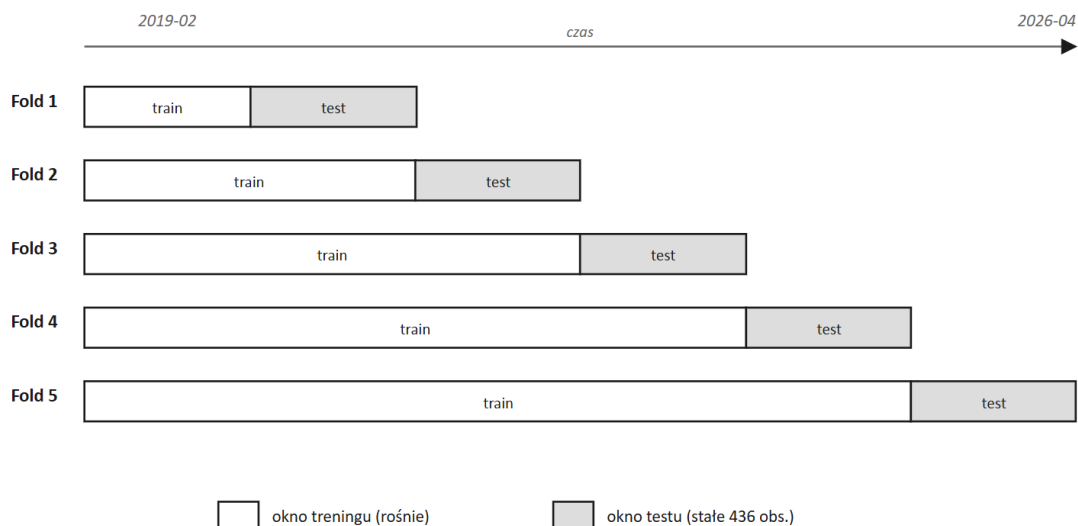
3.3 Implementacja budowy cech i targetu

Moduł inżynierii cech implementuje dziesięć cech zdefiniowanych w sekcji 1.2. Każda z cech jest realizowana przez osobną funkcję, działającą na obiekcie DataFrame i zwracającą nową kolumnę. Wszystkie operacje wykorzystują mechanizmy okien przesuwających (rolling windows) biblioteki pandas, które z definicji nie korzystają z informacji z przyszłości.

Po obliczeniu wszystkich cech, moduł konstruuje zmienną objaśnianą y zgodnie ze wzorem z sekcji 1.3, dla horyzontu 5 dni. Operacja ta wykorzystuje przesunięcie szeregu o -5, co powoduje, że ostatnie 5 wierszy zbioru nie ma poprawnie zdefiniowanej etykiety. Wiersze te są następnie usuwane operacją dropna(). W rezultacie po dodaniu cech i konstrukcji etykiety zbiór zawiera 2618 wierszy użytecznych, które stanowią podstawę dalszego modelowania. Dodatkowo moduł oznacza każdą obserwację etykietą reżimu rynkowego.

3.4 Schemat walidacji czasowej w kodzie

Moduł walidacji walk-forward stanowi rdzeń metodyczny pracy. Realizuje on procedurę walidacji z pięcioma foldami chronologicznymi opisaną w sekcji 1.4. Kluczowym elementem implementacji jest opakowanie każdego cyklu (skalowanie cech + trening modelu) w obiekt klasy Pipeline biblioteki scikit-learn. Pipeline gwarantuje, że metoda fit() skamera StandardScaler jest wywoływana wyłącznie na zbiorze treningowym bieżącego foldu, a następnie zastosowana metoda transform() do zbioru testowego.



*Każdy fold trenuje się tylko na danych chronologicznie wcześniejszych niż test.
Skaler dopasowywany wyłącznie na trainie bieżącego foldu (Pipeline scikit-learn).*

Rysunek 5. Schemat walidacji walk-forward z pięcioma foldami chronologicznymi

Dla każdego foldu moduł rejestruje cztery metryki klasyfikacyjne (accuracy, F1, ROC-AUC, log-loss), wektor surowych prawdopodobieństw klasy pozytywnej oraz wektor decyzji binarnych. Wyniki ze wszystkich pięciu foldów są agregowane do średniej i odchylenia standardowego, które pełnią rolę punktowej oceny jakości modelu. Surowe prawdopodobieństwa są zapisywane do plików Parquet, co pozwala na ich późniejsze wykorzystanie w modułach kalibracji, strojenia prognozy i analizy strategii inwestycyjnej.

3.5 Modele bazowe i porównywane modele

W pracy porównano pięć modeli klasyfikacyjnych, dobranych tak, aby reprezentować stopniowo rosnącą złożoność. Pierwsze dwa to baseline'y persistence - trywialne predyktory, w których prognoza dla kolejnego okresu jest powtórzeniem kierunku bieżącego zwrotu. W wariancie persistence-1d prognoza opiera się na zwrocie z poprzedniego dnia, w wariancie persistence-5d - na zwrocie z poprzednich pięciu dni, dopasowanym horyzontowo do wybranego zadania. Tak skonstruowane baseline'y pozwalają stwierdzić, czy dany model uczenia maszynowego dodaje rzeczywistą wartość do trywialnej heurystyki, czy jedynie kopiuje informację o autokorelacji zwrotów [1].

Trzecim modelem jest regresja logistyczna (ang. logistic regression), klasyczny liniowy klasyfikator, w którym prawdopodobieństwo klasy pozytywnej jest modelowane jako

sigmoida liniowej kombinacji cech. Mimo prostoty matematycznej, regresja logistyczna jest dobrym punktem odniesienia dla bardziej złożonych modeli, ponieważ pozwala stwierdzić, czy zależności między cechami a etykietą są wyraźnie nieliniowe.

Czwartym modelem jest las losowy (ang. random forest), zaproponowany przez Leo Breimana ensemble głębokich drzew decyzyjnych. Las losowy redukuje wariancję predykcji pojedynczych drzew poprzez agregację wyników wielu modeli wytrenowanych na próbkach z powtórzeniem (bootstrap) oraz z losowym podzbiorem cech. Algorytm jest odporny na overfitting, dobrze radzi sobie z cechami o różnej skali i nie wymaga jawnej regularyzacji.

Piątym modelem jest XGBoost (ang. eXtreme Gradient Boosting), nowoczesna implementacja gradient boosting drzew decyzyjnych. W odróżnieniu od lasu losowego, w którym drzewa są trenowane niezależnie, w gradient boosting każde kolejne drzewo jest trenowane tak, aby korygować błędy predykcji poprzedniego ensemble'u. XGBoost charakteryzuje się wysoką skutecznością w wielu zastosowaniach praktycznych, jednocześnie wymaga starannego doboru hiperparametrów.

3.6 Strojenie hiperparametrów

Dla trzech modeli uczenia maszynowego (regresja logistyczna, las losowy, XGBoost) przeprowadzono strojenie hiperparametrów z wykorzystaniem ograniczonej siatki 21 konfiguracji, dobranych na podstawie zaleceń w pracy Jansena [1]. Strojenie przeprowadzono w taki sam sposób, jak ocenę podstawową - z wykorzystaniem walidacji walk-forward, tak aby uniknąć wycieku danych z przyszłości. Tabela 2 przedstawia najlepsze konfiguracje wraz z odpowiednimi średnimi ROC-AUC i odchyleniami standardowymi.

Tabela 2. Najlepsze konfiguracje hiperparametrów po strojeniu

Model	Najlepsza konfiguracja	ROC-AUC (średnia)	ROC-AUC (std)
Regresja logistyczna	$C = 0,1$	0,5334	0,0057
Las losowy	$\text{max_depth} = 5,$ $\text{min_samples_leaf} = 10$	0,5496	0,0343

XGBoost	max_depth = 3, learning_rate = 0,03	0,5442	0,0280
---------	--	--------	--------

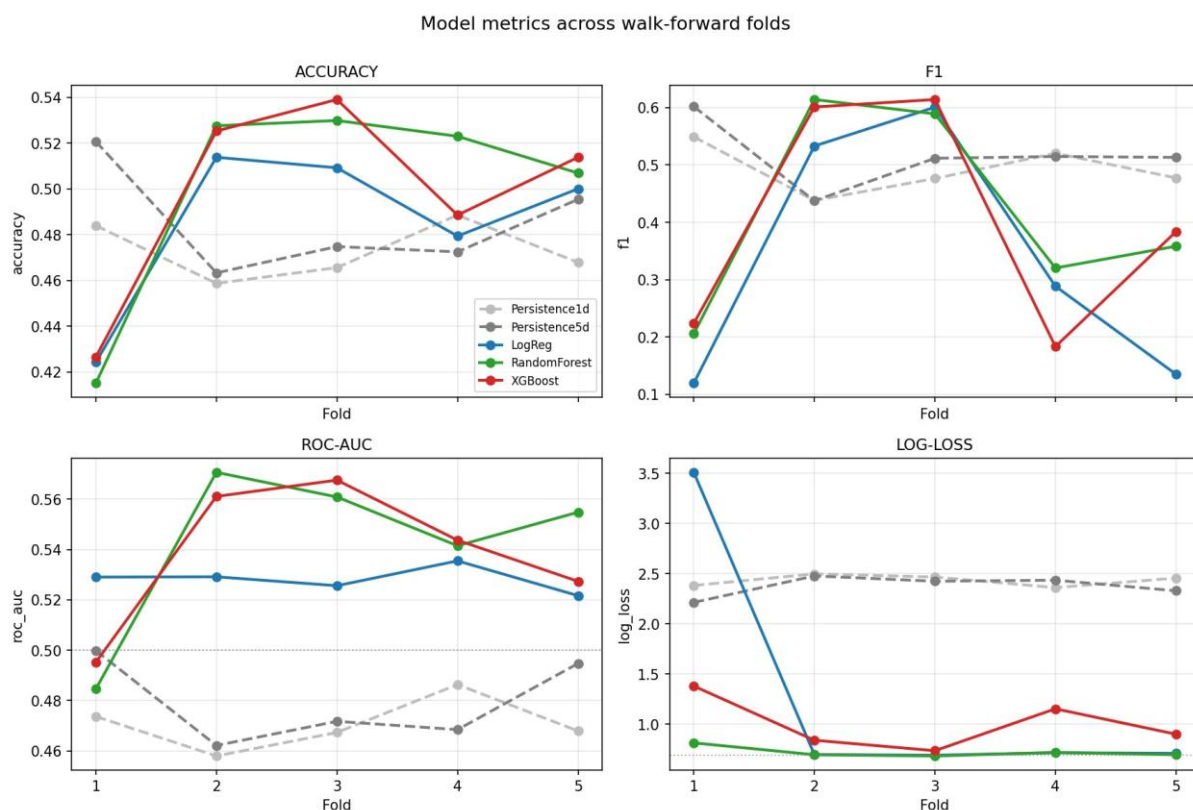
Najbardziej znaczącą obserwacją jest fakt, że strojenie hiperparametrów daje stosunkowo niewielkie przyrosty ROC-AUC - rzędu 0,5-1 punktu procentowego w porównaniu z konfiguracjami domyślnymi. Wszystkie najlepsze konfiguracje są płytkie i silnie regularyzowane (mała głębokość drzew, niski współczynnik C dla regresji logistycznej, niskie tempo nauczania dla XGBoost), co odzwierciedla niski stosunek sygnału do szumu w danych finansowych. Głębokie modele uczyłyby się szumu jako sygnału, czyli klasycznie overfittowały

3.7 Porównanie modeli

Tabela 3 przedstawia wyniki podstawowe pięciu porównywanych modeli, obliczone jako średnia z pięciu foldów walk-forward. Wszystkie modele oceniono w czterech metrykach przy domyślnym progu decyzyjnym 0,5 i bez stosowania kalibracji.

Tabela 3. Wyniki podstawowe pięciu modeli (średnia z 5 foldów walk-forward)

Model	Accuracy	F1	ROC-AUC	Log-loss
Persistence-1d	0,473	0,492	0,471	2,432
Persistence-5d	0,485	0,516	0,479	2,375
Regresja logistyczna	0,485	0,335	0,528	1,264
Las losowy	0,501	0,417	0,543	0,721
XGBoost	0,499	0,401	0,539	1,002



Rysunek 6. Stabilność metryk per fold - rozproszenie wyników między foldami walk-forward

Najważniejszych wniosków z Tabeli 3 jest trzy:

- **Modele uczenia maszynowego przewyższają oba baseline'y persistence w metryce ROC-AUC.** Najlepszy model (las losowy) osiąga ROC-AUC 0,543, podczas gdy najlepszy baseline (persistence-5d) - 0,479. Różnica około sześciu punktów procentowych jest istotna w kontekście danych finansowych, gdzie wartości AUC dla horyzontu dziennego, na podstawie literatury [1], zwykle pozostają w przedziale 0,51-0,55.
- **Wszystkie modele uczenia maszynowego mają zauważalnie niższą miarę F1 niż baseline'y persistence przy progu 0,5.** Paradoks ten wynika z konserwatywności modeli ML - przy domyślnym progu wykrywają one mniej obserwacji klasy pozytywnej, co obniża F1 mimo prawidłowego rankowania. Obserwacja ta bezpośrednio motywuje strojenie progu decyzyjnego, omówione w sekcji 3.8.
- **Las losowy osiąga najniższe log-loss (0,721).** Niska wartość log-loss przy umiarkowanej dokładności wskazuje, że prawdopodobieństwa wystawiane przez ten model są lepiej skalibrowane niż w pozostałych modelach, co jest naturalną konsekwencją techniki bagging.

3.8 Kalibracja prawdopodobieństw i strojenie progu decyzyjnego

Klasyfikator uczenia maszynowego zwraca w fazie wnioskowania dwa rodzaje wyjść: dyskretną decyzję klasyfikacyjną (klasa 0 lub 1) oraz ciągłe prawdopodobieństwo przynależności do klasy pozytywnej. Decyzja dyskretna otrzymywana jest przez porównanie prawdopodobieństwa z arbitralnie ustalonym progiem (domyślnie 0,5). W tej sekcji przedstawiono wyniki dwóch rozszerzeń metodycznych pracy: kalibracji izotonicznej prawdopodobieństw oraz strojenia progu decyzyjnego.

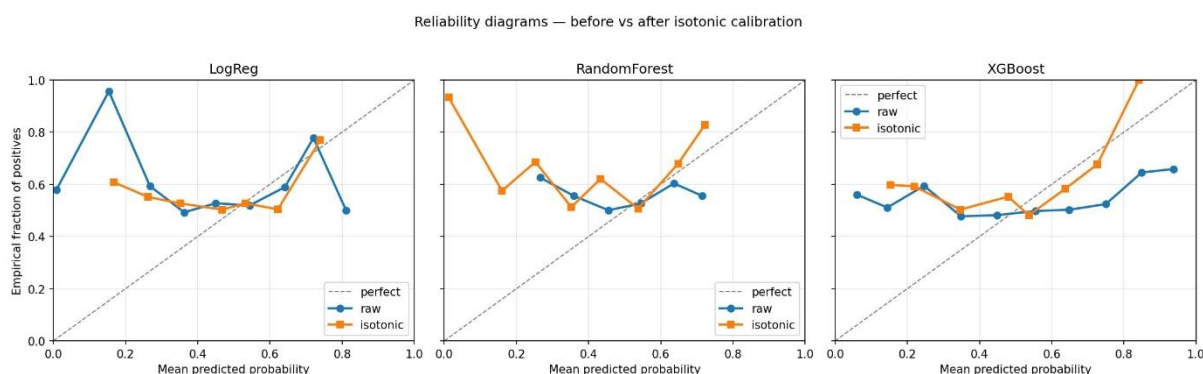
3.8.1 Kalibracja izotoniczna

Mówimy, że klasyfikator jest dobrze skalibrowany, jeżeli wśród obserwacji, dla których model przewidział prawdopodobieństwo p , faktyczna częstotliwość klasy pozytywnej wynosi w przybliżeniu p . Metoda kalibracji izotonicznej, zaproponowana przez Niculescu-Mizila i Caruanę [3], jest nieparametryczną procedurą stosowaną po wytrenowaniu modelu, która mapuje surowe prawdopodobieństwa na funkcję monotonicznie rosnącą, lepiej oddającą rzeczywistą częstotliwość klasy pozytywnej.

Modułem calibration.py zaaplikowano kalibrację izotoniczną do trzech modeli uczenia maszynowego. Tabela 4 przedstawia log-loss przed i po kalibracji wraz z procentową zmianą.

Tabela 4. Wpływ kalibracji izotonicznej na log-loss

Model	Log-loss bez kalibracji	Log-loss po kalibracji	Zmiana
Regresja logistyczna	1,264	0,784	-37,9%
Las losowy	0,721	0,869	+20,5%
XGBoost	1,002	0,759	-24,3%



Rysunek 7. Krzywe niezawodności modeli przed i po kalibracji izotonicznej

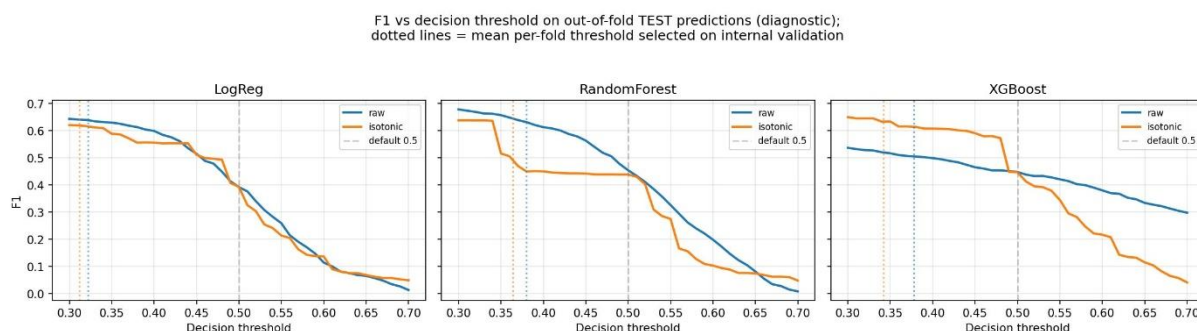
Najbardziej zaskakującym wynikiem jest fakt, że kalibracja izotoniczna pogorszyła log-loss lasu losowego o ponad dwadzieścia procent. Wynik ten ma jednak prostą interpretację teoretyczną: las losowy, dzięki technice bagging, jest naturalnie dobrze skalibrowany. Aplikacja kalibracji izotonicznej do już skalibrowanego modelu wprowadza dodatkowy szum z procesu dopasowania krzywej izotonicznej. Z drugiej strony, regresja logistyczna i XGBoost, których prawdopodobieństwa były istotnie zniekształcone, drastycznie zyskały na kalibracji - log-loss XGBoost spadł o 24%, a log-loss regresji logistycznej - o niemal 38%. Głównym wnioskiem metodycznym z tej analizy jest, że kalibracja prawdopodobieństw powinna być stosowana selektywnie.

3.8.2 Strojenie progu decyzyjnego

Strojenie progu decyzyjnego (ang. threshold tuning) jest drugą, prostą procedurą, która potrafi mocno zmienić jakość predykcji bez modyfikacji samego modelu. W warunkach niskiego stosunku sygnału do szumu, charakterystycznego dla danych finansowych, modele klasyfikacyjne często są zbyt konserwatywne przy progu 0,5. Aby uniknąć doboru progu na danych testowych (co prowadziłoby do przeszacowania wyniku), próg dobierany jest niezależnie wewnątrz każdego foldu walk-forward: na wydzielonej wewnętrznej walidacji obejmującej ostatnie 20% okna treningowego danego foldu przegląda się 41 wartości progu w zakresie [0,30; 0,70] z krokiem 0,01 i wybiera tę maksymalizującą miarę F1. Tak dobrany próg jest następnie stosowany do predykcji testowych tego foldu. Zbiór testowy nigdy nie uczestniczy w wyborze progu. Wartość raportowana w tabeli to średnia z pięciu progów dobranych w poszczególnych foldach.

Tabela 5. Wpływ strojenia progu decyzyjnego na miarę F1

Model	F1 (próg 0,5)	Optymalny próg	F1 (próg stuningowany)	Poprawa
Regresja logistyczna	0,392	0,312	0,619	+57,9%
Las losowy	0,454	0,364	0,608	+33,9%
XGBoost	0,447	0,342	0,633	+41,6%



Rysunek 8. Zależność miary F1 od progu decyzyjnego

Po zastosowaniu poprawnej procedury doboru progu wewnątrz foldów, optymalne progi dla trzech modeli mieszczą się w przedziale od 0,31 do 0,36 (średnia z pięciu foldów), a poprawa miary F1 względem progu domyślnego 0,5 wynosi od 34 do 58 procent. Co ważne, progi nie są już przyklejone do dolnej granicy zakresu, jak działało się przy wcześniejszym doborze na zbiorze testowym, lecz przyjmują wartości wewnętrzne dobrane na danych walidacyjnych. Analiza progów w poszczególnych foldach pokazuje też ciekawą zależność: w foldzie obejmującym bessę i odbicie z lat 2022-2023 wszystkie trzy modele potrzebują wyraźnie wyższego progu (0,51-0,57), co można odczytać jako niezależny sygnał zmiany reżimu rynkowego. Wniosek metodyczny jest taki, że dobór progu decyzyjnego wpływa na miarę F1 silniej niż wybór algorytmu, a tym bardziej niż strojenie hiperparametrów w obrębie tej samej rodziny modeli.

Wybór progu decyzyjnego ma w klasyfikacji binarnej z niskim stosunkiem sygnału do szumu większy wpływ na ostateczną miarę F1 niż wybór samego algorytmu, a tym bardziej niż strojenie hiperparametrów w obrębie tej samej rodziny modeli.

3.9 Analiza stabilności w czasie i typowe błędy

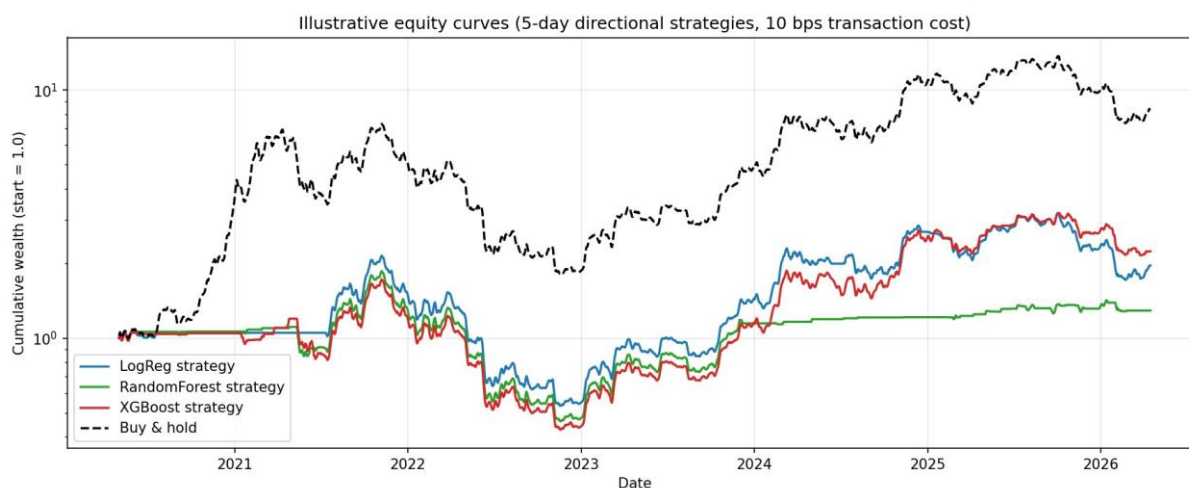
W tej sekcji przedstawiono dwa rozszerzenia metodyczne pozwalające ocenić, czy obserwowana jakość modelu jest stabilna w czasie: uproszczoną symulację strategii inwestycyjnej z kosztami transakcyjnymi oraz analizę wyników w pięciu ręcznie zdefiniowanych reżimach rynkowych.

3.9.1 Uproszczona symulacja strategii inwestycyjnej

Aby ocenić praktyczne znaczenie predykcji modelu, przeprowadzono uproszczoną symulację strategii inwestycyjnej typu long-only. Symulacja zakłada, że inwestor otwiera długą pozycję w Bitcoinie każdorazowo, gdy skalibrowane prawdopodobieństwo modelu przekracza próg decyzyjny dobrany dla danego foldu (wartości w zakresie 0,31-0,36), oraz zamyka pozycję w przypadku przeciwnym. Każda zmiana pozycji wiąże się z kosztem transakcyjnym wynoszącym 10 punktów bazowych (0,10%), co odpowiada przybliżonej wartości prowizji na giełdach kryptowalut.

Tabela 6. Wyniki symulacji strategii inwestycyjnej

Strategia	Liczba transakcji	Hit rate	Sharpe	Max DD	Final equity
Regresja logistyczna	104	0,516	0,45	-75%	1,96x
Las losowy	109	0,518	0,19	-75%	1,29x
XGBoost	179	0,525	0,51	-75%	2,24x
Kup-i-trzymaj	1	-	-	-77%	8,41x



Rysunek 9. Krzywe kapitału strategii ML i strategii kup-i-trzymaj

Z symulacji wynikają dwie rzeczy. Po pierwsze, wszystkie trzy strategie ML mają dodatni współczynnik Sharpe'a i hit rate powyżej 50%, więc sygnał w predykcjach jest realny. Po drugie, mimo to każda z nich przegrywa pod względem łącznej stopy zwrotu z prostą strategią kup-i-trzymaj, która kończy z kapitałem rzędu 8,4x wobec 2-3x dla strategii ML. Wynika to z silnego trendu wzrostowego Bitcoina w badanym okresie, w którym strategia siedząca część czasu w gotówce traci ekspozycję na wzrosty.

3.9.2 Stabilność modeli per reżim rynkowy

Ostatnim rozszerzeniem standardowej ewaluacji jest analiza stabilności wyników modeli w pięciu ręcznie zdefiniowanych reżimach rynkowych. Tabela 7 przedstawia ROC-AUC każdego modelu w każdym reżimie.

Tabela 7. ROC-AUC modeli w pięciu reżimach rynkowych

Reżim	LogReg	RF	XGBoost	Persistence-5d
pre_2020 (n=30)	0,727	0,528	0,611	0,333
bull_2020_2021 (n=548)	0,500	0,476	0,483	0,510
bear_2022 (n=396)	0,525	0,575	0,594	0,435
recovery_2023 (n=365)	0,547	0,556	0,553	0,473
bull_2024_plus (n=841)	0,539	0,547	0,532	0,487

Najciekawszym wynikiem analizy jest to, że modele ML zachowują się zupełnie inaczej w różnych reżimach rynkowych. W okresie bessy 2022, z wyraźnym trendem spadkowym, modele drzewiaste osiągają swoje najlepsze wyniki - XGBoost dochodzi do ROC-AUC 0,594. Z kolei w okresie parabolicznej hossy 2020-2021 wszystkie modele osiągają wyniki bliskie losu (0,48-0,50). Wniosek: pojedyncza, uśredniona miara jakości modelu zacierza duże różnice w jego zachowaniu w różnych warunkach rynkowych.

3.10 Podsumowanie części praktycznej, ograniczenia i dalsze kroki

Przeprowadzone eksperymenty pozwalają na sformułowanie sześciu kluczowych wniosków. Po pierwsze, klasyczne modele uczenia maszynowego osiągają wyraźną przewagę nad trywialnymi predyktorami persistence w metryce ROC-AUC. Po drugie, paradoksalnie, te same modele wypadają gorzej od baseline'ów persistence w metryce F1 przy domyślnym progu decyzyjnym 0,5, co wynika z ich konserwatywności w warunkach niskiego sygnału. Po trzecie, strojenie progu decyzyjnego, przeprowadzone bez dotykania zbioru testowego, daje przyrosty F1 rzędu 34-58%, znacznie przewyższające przyrosty ze strojenia hiperparametrów lub wyboru algorytmu. Po czwarte, kalibracja izotoniczna prawdopodobieństw poprawia log-loss regresji logistycznej i XGBoost o 24-38%, ale pogarsza ten wskaźnik dla lasu losowego. Po piąte, mimo dodatniego współczynnika Sharpe'a wszystkie strategie ML pozostają zdominowane ekonomicznie przez prostą strategię kup-i-trzymaj. Po szóste, wyniki modeli są wyraźnie niestabilne między reżimami rynkowymi, co widać między innymi po tym, że dobierany próg decyzyjny rośnie w okresie bessy.

Najważniejszym spostrzeżeniem z perspektywy inżynierskiej jest fakt, że inwestycja w jakość procedury walidacji - eliminacja wycieku danych z przyszłości, opakowanie w Pipeline, niezależna kalibracja - daje wyraźniejsze przyrosty jakości niż wybór bardziej zaawansowanego algorytmu. W praktyce różnica między regresją logistyczną a XGBoost wynosi około jednego punktu procentowego ROC-AUC, podczas gdy różnica między uczciwie a nieuczciwie zwalidowanym XGBoost może osiągać kilkanaście punktów procentowych.

Ograniczenia. Praca podlega kilku istotnym ograniczeniom o charakterze technicznym i metodologicznym. Po pierwsze, opiera się wyłącznie na dziennych danych OHLCV - dane intraday zapewniłyby od 24 do 288 razy większą liczbę obserwacji. Po drugie, analiza jest ograniczona do pary BTC/USDT. Po trzecie, symulacja strategii inwestycyjnej jest świadomie uproszczona i nie modeluje mikrostruktury rynku, slippage'u ani zarządzania portfelem. Po czwarte, praca nie zawiera formalnych testów istotności statystycznej.

Dalsze kroki. Zaproponowane w pracy podejście otwiera kilka naturalnych kierunków dalszych badań: wykorzystanie danych intraday i modeli sekwencyjnych typu LSTM lub Transformer; predykcja punktów wyjścia z pozycji jako osobnego zadania ML; detekcja okresów niskiej pewności predykcji poprzez wykorzystanie wewnętrznej pewności modelu jako sygnału filtrującego; integracja danych alternatywnych (sentiment, on-chain); online learning adaptujący model do zmian reżimu rynkowego.

Podsumowanie

Celem niniejszej pracy było zaprojektowanie, implementacja i empiryczna ewaluacja kompletnego potoku przetwarzania danych do prognozowania kierunku zmian ceny Bitcoina w horyzoncie pięciu dni, opartego na klasycznych metodach uczenia maszynowego nadzorowanego. Cel ten został osiągnięty.

W ramach pracy zaimplementowano w pełni reprodukowalny pipeline obejmujący pobieranie danych z giełdy Binance, inżynierię dziesięciu cech, walidację walk-forward z pięcioma foldami chronologicznymi, oraz trening i porównanie pięciu modeli klasyfikacyjnych. Standardowa procedura została rozszerzona o cztery elementy metodyczne podnoszące wartość praktyczną pracy: kalibrację izotoniczną prawdopodobieństw, strojenie prognozy

decyzyjnego, uproszczoną symulację strategii inwestycyjnej z kosztami transakcyjnymi oraz analizę stabilności w pięciu ręcznie zdefiniowanych reżimach rynkowych. Wszystkie eksperymenty zostały zarejestrowane w narzędziu MLflow w hierarchicznej strukturze przebiegów nadrzędnych i podrzędnych (parent-child).

Najważniejszy wynik nie jest liczbowy, lecz metodyczny: w danych o tak niskim stosunku sygnału do szumu – typowych dla dziennych danych finansowych – o końcowej jakości decyduje rzetelność walidacji i procedur po treningu (kalibracja, dobór progu), a nie wybór algorytmu. Sam dobór progu decyzyjnego, wykonany bez zaglądania do zbioru testowego, podniósł miarę F1 o 34%-58%, podczas gdy całe strojenie hiperparametrów dało zaledwie 0,5-1 punktu procentowego ROC-AUC. Podobnie różnica między ROC-AUC między najprostszą regresją logistyczną a XGBoost wynosi około jednego punktu procentowego – natomiast różnica między tym samym modelem zwalidowanym uczciwie i nieuczciwie potrafi sięgać kilkunastu punktów procentowych ROC-AUC.

Wyniki ilościowe są świadomie skromne – ROC-AUC około 0,54 i ekonomiczna przegrana z buy-and-hold w okresie silnego trendu wzrostowego – ale są powtarzalne, wolne od wycieku danych z przyszłości i mogą służyć jako rzetelny punkt odniesienia do dalszych badań. Główny wniosek pracy daje się sprowadzić do jednego zdania: w finansowym uczeniu maszynowym na danych dziennych uczciwa inżynieria zwraca się szybciej niż złożoność modelu.

Bibliografia

- [1] S. Jansen, Machine Learning for Algorithmic Trading, wyd. 2, Birmingham: Packt Publishing, 2020.
- [2] M. López de Prado, Advances in Financial Machine Learning, Hoboken: Wiley, 2018.
- [3] A. Niculescu-Mizil i R. Caruana, „Predicting Good Probabilities with Supervised Learning”, w Proc. of 22nd International Conference on Machine Learning (ICML 2005), Bonn, 2005, pp. 625-632. (cytowany za Jansenem [1])
- [4] scikit-learn developers, „scikit-learn: Machine Learning in Python - User Guide”, https://scikit-learn.org/stable/user_guide.html, (dostęp: 25.04.2026).

[5] Binance, „Binance API Documentation”, <https://binance-docs.github.io/apidocs/spot/en/>, (dostęp: 25.04.2026).

[6] MLflow, „MLflow Documentation”, <https://mlflow.org/docs/latest/index.html>, (dostęp: 25.04.2026).

Spis rysunków

Rysunek 1. Cena Bitcoina w analizowanym okresie z zaznaczonymi reżimami rynkowymi

Rysunek 2. Struktura repozytorium btc-ml-pipeline

Rysunek 3. Diagram pipeline'u uczenia maszynowego

Rysunek 4. Struktura śledzenia eksperymentów w MLflow

Rysunek 5. Schemat walidacji walk-forward z pięcioma foldami chronologicznymi

Rysunek 6. Stabilność metryk per fold - rozproszenie wyników między foldami walk-forward

Rysunek 7. Krzywe niezawodności modeli przed i po kalibracji izotonicznej

Rysunek 8. Zależność miary F1 od progu decyzyjnego

Rysunek 9. Krzywe kapitału strategii ML i strategii kup-i-trzymaj

Spis tabel

Tabela 1. Główne biblioteki wykorzystane w pracy

Tabela 2. Najlepsze konfiguracje hiperparametrów po strojeniu

Tabela 3. Wyniki podstawowe pięciu modeli (średnia z 5 foldów walk-forward)

Tabela 4. Wpływ kalibracji izotonicznej na log-loss

Tabela 5. Wpływ strojenia progu decyzyjnego na miarę F1

Tabela 6. Wyniki symulacji strategii inwestycyjnej

Tabela 7. ROC-AUC modeli w pięciu reżimach rynkowych